

Soleux Control API

TCP Default + 3 and HTTP/HTTPS Command Specification

Status: implemented protocol 2 command catalogue for all five active device profiles

Audience: Device firmware, Windows application and mobile application developers

Transports: TCP legacy port + 3 (5008 by default), HTTP port 80 and HTTPS port 443

Compatibility: migrated device base ports are AT-only; Windows device paths use the Control API

Implementation status: Relay Module, AC/DC Dimmer, PDU Energy Meter, High Power PDU and PDU V1 firmware expose the implemented protocol 2 command subsets on the configured base TCP port plus 3 and through POST /api/v1/command over HTTP or HTTPS. This specification lists only those implemented actions; device-specific actions remain capability-gated.

Document purpose

This specification defines the implemented transport-neutral command model for Soleux device firmware, the Windows application and mobile clients. Migrated device control uses the dedicated Control API transport; the configured base port remains available for legacy AT clients.

Normative rules

1. All physical input, virtual-input and output indexes are zero-based on every transport.
2. JSON booleans are true/false; API clients must not send 0/1 as boolean substitutes.
3. Time durations use integer milliseconds unless a field name explicitly ends in `_s`.
4. Electrical and environmental units are included in field names: `_v`, `_a`, `_w`, `_va`, `_wh`, `_hz` and `_c`.
5. Timestamps use ISO 8601 with an offset. Event timestamps should be UTC where practical.
6. Unknown fields may be ignored only when the negotiated protocol version permits forward-compatible extensions.
7. Secrets are write-only unless a command explicitly documents that they can be returned.
8. Every state-changing command identifies its required permission and produces an auditable source/session identity.

Command groups

- Session and identity: 4 implemented actions.
- Core device state: 4 implemented actions.
- Input control: 3 implemented actions on supported profiles.
- Output control: 3 implemented actions on all profiles.
- Mapping and hardware control: 2 implemented actions on supported profiles.
- Dimmer control and output groups: 6 implemented actions on the AC/DC Dimmer.
- Energy metering: 2 implemented actions on the PDU Energy Meter.
- Configuration pages: 4 implemented actions on all profiles.
- Transfers: 7 implemented actions on all profiles.

Catalogue size: 31 implemented request actions.

Transport and message envelope

The action and params object is identical on TCP, HTTP and HTTPS. Each HTTP request is dispatched through the same command handler used by the newline-delimited TCP service, so command parameters and result envelopes do not change between transports.

Request envelope

Field	Type	Required	Constraints/default	Description
protocol	integer	Yes	2 for current Relay; 3 for target contract	Protocol version.
id	integer string	Yes	Unique among pending requests	Caller-generated correlation ID.
action	string	Yes	Exact action name	Command to execute.
params	object	No	Default {}	Command-specific parameters.
timeout_ms	integer	No	100-120000	Optional caller timeout hint; does not override safety limits.

```
{"protocol":2,"id":42,"action":"set_output_state","params":{"channel":0,"state":true}}
```

Success response

Field	Type	Presence	Description
protocol	integer	Always	Protocol version.
id	integer string	Always	Original request ID.
ok	boolean	Always	True for a successful command.
result	object	Always	Command-specific expected result.

```
{"protocol":2,"id":42,"ok":true,"result":{"channel":0,"requested_state":true,"actual_state":true,"pending":false}}
```

Error response

Field	Type	Presence	Description
protocol	integer	Always	Protocol version.
id	integer string null	Always	Original ID, or null when it could not be parsed.
ok	boolean	Always	False for an error.
error.code	string	Always	Stable machine-readable error code.

Field	Type	Presence	Description
error.message	string	Always	Safe human-readable explanation.
error.details	object	Optional	Structured validation or recovery data.
error.retryable	boolean	Always	Whether retry may succeed without changing the request.

```
{"protocol":2,"id":42,"ok":false,"error":{"code":"invalid_request","message":"output channel out of range"}}
```

Transport mapping

Transport	Request framing	Response framing	Events
TCP: legacy port + 3	One UTF-8 JSON object per line (CRLF or LF). Default 5005 becomes Control API 5008.	One JSON response line using the common envelope.	Command responses only in the current Relay implementation.
HTTP port 80	POST /api/v1/command with application/json.	HTTP status plus the common JSON envelope.	No Control API SSE endpoint is implemented yet.
HTTPS port 443	Same endpoint over TLS. When SSL is enabled, HTTP receives a 307 redirect that preserves POST and body.	Same common JSON envelope.	No secure event endpoint is implemented yet.
WebSocket / SSE	Reserved for a future event transport.	Not currently a Control API command transport.	Planned; capability-dependent.

Implemented HTTP/HTTPS endpoint

Each migrated device family exposes one transport-neutral command endpoint. Every action available on that device's JSON TCP Control API is accepted at POST /api/v1/command, including state/configuration operations, page configuration and actions, system-log pagination and deletion, and supported file-transfer operations.

Property	Implemented behavior
Endpoint	POST /api/v1/command
Media type	Request and response use application/json; the response also sets Cache-Control: no-store.
Request body	One JSON object using the common protocol, id, action and params fields.
Maximum body	1,048,576 bytes. The limit is enforced with or without a Content-Length header.
Dispatch	The request is passed to the same process_tcp_json dispatcher used by the TCP Control API.
Client access	api_enabled must be true. Existing HTTP API IP/MAC rules then apply to the direct socket peer; forwarding headers are not trusted for authorization.
HTTP/HTTPS	The same Flask endpoint is served on HTTP 80 when SSL is off and HTTPS 443 when SSL is on.

Property	Implemented behavior
Legacy API	The existing /cmdapi endpoint is unchanged and remains separate from this JSON command endpoint.

HTTP status mapping

Status	Meaning	Response
200	The command completed successfully.	ok is true and result contains the command output.
400	Malformed body, invalid parameters or unknown action.	ok is false with invalid_request, unknown_action or another command error.
403	The peer is not allowed by the configured API IP/MAC rules.	ok is false with unauthorized.
413	The request body exceeds 1 MiB.	ok is false with message_too_large.
503	The HTTP/HTTPS Control API is disabled.	ok is false, id is null and error.code is api_disabled.
504	An asynchronous device or hardware operation timed out.	ok is false with timeout.
500	The dispatcher failed internally or produced an invalid response.	ok is false with internal_error.

HTTP request example

```
POST /api/v1/command HTTP/1.1
Content-Type: application/json

{"protocol":2,"id":"mobile-1","action":"hello","params":{}}

{"protocol":2,"id":"mobile-1","ok":true,"result":{"device":"dimmer","api_port":5008}}
```

Verification commands

Run the source-level contract tests from the Soleux App project root on a developer computer with Python 3. Each active device family has its own tests folder; the Dimmer command contract is verified with:

```
python -m unittest discover -s "Device Firmware\Dimmer\tests" -p "test_*.py" -v
```

After deploying the matching firmware files and restarting WebService, run this on the device to verify the live HTTP endpoint:

```
curl -sS -X POST http://127.0.0.1/api/v1/command \
-H 'Content-Type: application/json' \
-d '{"protocol":2,"id":"test-1","action":"hello","params":{}}'
```

Use `https://127.0.0.1/api/v1/command` when SSL is enabled. A successful response has `ok: true` and reports the calculated `api_port`.

Compatibility rule: The `J:` prefix is not accepted on the Control API or migrated base ports. Standard Windows paths use unprefixed JSON on base port + 3 and do not send AT commands or fall back to the AT-only base port. Adding the HTTP/HTTPS endpoint does not change existing legacy AT behavior.

Permissions and security

Permission	Allows
Public	hello only.
Read	Read identity, state, configuration summaries, telemetry and events.
Control	Change live input/output/dimmer state and execute schedules/scenarios.
Configure	Change mappings, channel configuration, schedules, automations and non-secret settings.
Admin	Accounts, access rules, network/security settings, backup/restore, firmware, reboot and destructive actions.

The implemented HTTP/HTTPS command endpoint first requires `api_enabled`. When enabled, each device's existing API IP/MAC allow rules apply to the direct socket peer. Disabling filtering allows any reachable peer; it is different from disabling the API service. Forwarding headers are not trusted. Restrict HTTP to trusted local networks and use HTTPS for credentials, backup/restore, firmware, or remote/mobile access. Tokens and secrets must never be written to normal logs.

Implemented service access switches

All five active firmware families persist the following independent Security-page switches. Both default to enabled. Changing either switch schedules a WebService restart after the current response is delivered.

Security field	Behavior when disabled	Still available and recovery
<code>tcp_enabled</code>	Closes and suppresses the base TCP, base + 1 and Control API TCP base + 3 listeners after restart. Existing TCP sessions close and the Windows app cannot connect.	UDP discovery, UDP heartbeat base + 2 and the web UI remain available. Re-enable TCP from the web Security page.
<code>api_enabled</code>	HTTP/HTTPS command routes return status 503 with <code>id null</code> and <code>error.code api_disabled</code> . Raw Control API TCP is not closed.	The normal web UI remains available. Re-enable API from the web Security page.

Disabling API does not close Control API TCP; disabling TCP does. If both are disabled, the web Security page remains the recovery path. Discovery or heartbeat success therefore does not prove that device control is enabled.

```
{"protocol":2,"id":null,"ok":false,"error":{"code":"api_disabled","message":"HTTP/HTTPS API access is disabled."}}
```

Implemented AC/DC Dimmer profile

The active Dimmer firmware accepts unprefixed protocol 2 JSON on `base_port + 3` (normally 5008) and the identical command object through `POST /api/v1/command` over HTTP or HTTPS. The base port remains AT-only and rejects J-prefixed traffic.

All Dimmer input, output and mapping indices are zero-based. `set_dimmer_level` requires channel and an integer value from 0 through 100. `set_output_state` and `restart_output` use the same zero-based channel rule. Output-group master and local-slave channels are also zero-based; remote `output_number` values are one-based. A remote slave stores its Dimmer base port, and PWM synchronization uses that port plus 1.

`get_device_state` returns requested `set_pwm` and live `actual_pwm` for every output, plus I/O state, PWM frequency, sensors, system and network data. Clients should poll this snapshot when a state-changing result reports pending: true.

The Dimmer also implements device identity, relay configuration, input/output configuration, mappings, hardware write, output groups, settings/security/system/schedule/automation/scenario/sequence pages, chunked backup/restore, paginated system logs and `execute_page_action delete_system_logs`.

Discovery advertises `PORT`, `API_PORT`, `API_VER` and `CAPS`. Heartbeat uses `base_port + 2` and reports `tcp_port`, `api_port` and `api_version`. Both offsets are calculated from the configured base port.

Common error catalogue

Every error uses the common error envelope. Commands may document additional codes.

Code	Meaning
<code>invalid_request</code>	Envelope or JSON structure is invalid.
<code>unknown_action</code>	Action name is not recognized.
<code>unsupported_command</code>	Device recognizes the action but does not support it.
<code>unsupported_protocol</code>	No mutually supported protocol version exists.
<code>authentication_required</code>	A valid authenticated session is required.
<code>authentication_failed</code>	Credentials or token were rejected.
<code>permission_denied</code>	Session lacks the required permission.

Code	Meaning
invalid_parameter	A parameter has the wrong type, range or value.
invalid_channel	Input/output channel is outside the zero-based device range.
not_found	Requested resource or operation does not exist.
revision_conflict	expected_revision does not match current resource revision.
busy	A conflicting operation is active.
interlock	A safety or control interlock prevented the operation.
timeout	The operation did not complete within its allowed time.
rate_limited	Caller exceeded the configured request limit.
payload_too_large	Message or transfer exceeds advertised limits.
internal_error	Unexpected device-side failure; message must not expose secrets.
api_disabled	The HTTP/HTTPS Control API is disabled in device Security settings.

Implemented command catalogue

This catalogue contains only request actions implemented by the active protocol-2 firmware profiles. Device-specific actions are listed with their applicable profile; clients should still use the hello capability list before sending a device-specific action.

Action	Implemented by	Parameters	Result or purpose
hello	All active profiles	None	Protocol, identity and capability list.
get_device_info	All active profiles	None	Stable identity, firmware and network information.
get_device_state	All active profiles	None	Live inputs, outputs, PWM, sensors, system and network snapshot.
get_relay_configuration	All active profiles	None	Input, output and mapping configuration.
set_input_state	Relay, Dimmer, PDU Energy, PDU 10 kW	channel, state, source	Inject a physical input state.
set_virtual_input_state	Relay, Dimmer, PDU Energy, PDU 10 kW	channel, state, source	Inject a virtual input state.
set_input_configuration	Relay, Dimmer, PDU Energy, PDU 10 kW	channel, name, enabled	Save one input configuration.
set_output_state	All active profiles	channel, state, source	Request an output on or off state.
restart_output	All active profiles	channel, source	Restart one output.

Action	Implemented by	Parameters	Result or purpose
set_output_configuration	All active profiles	Device-specific output fields	Save one output configuration.
set_mapping	Relay, Dimmer, PDU Energy, PDU 10 kW	input, output, code	Save one input-to-output mapping.
write_hardware	Relay, Dimmer, PDU Energy, PDU 10 kW	None	Publish saved mappings and output delays to hardware services.
set_dimmer_level	AC/DC Dimmer	channel, value (0-100)	Set one PWM brightness level.
set_dimmer_frequency	AC/DC Dimmer	value	Save and apply the dimmer frequency mode.
get_output_groups	AC/DC Dimmer	None	Read Dimmer output names and all local/remote output groups.
save_output_group	AC/DC Dimmer	group	Create or update one output group and publish its local mask.
delete_output_group	AC/DC Dimmer	group_id	Delete one output group and publish the revised local masks.
rewrite_output_groups	AC/DC Dimmer	None	Rewrite every local group mask to the MCU and wait for its acknowledgement.
get_energy_state	PDU Energy Meter	None	Read current aggregate energy measurements.
get_energy_history	PDU Energy Meter	parameter, start_date, end_date	Read stored energy history.
get_page_configuration	All active profiles	page, optional pagination	Read a supported configuration page.
set_page_configuration	All active profiles	page, section, values	Save a supported configuration page section.
mutate_page_row	All active profiles	page, section, operation, row_id, values	Create, update or delete a supported page row.
execute_page_action	All active profiles	page_action and action-specific fields	Execute supported system, time, log and reboot actions.
transfer_upload_begin	All active profiles	kind, role, filename, size	Start a chunked upload session.
transfer_upload_chunk	All active profiles	transfer_id, role, offset, data	Append one upload chunk.
transfer_upload_finish	All active profiles	transfer_id, role	Finish and validate an upload.
transfer_commit	All active profiles	transfer_id	Commit a validated upload or restore.
settings_backup_download_begin	All active profiles	None	Start a settings-backup download.
transfer_download_chunk	All active profiles	transfer_id, offset	Read one download chunk.

Action	Implemented by	Parameters	Result or purpose
transfer_download_finish	All active profiles	transfer_id	Finish a download session.

The catalogue intentionally excludes target or proposed actions that do not appear in the active firmware dispatcher and hello capability data.

1. Device session and state

1.1 hello

Negotiate protocol 2 and return the device identity, installed channel counts, API port and advertised capabilities.

Implemented by: All active profiles **Operation:** Public read

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
protocol	integer	Always	Current Control API protocol number, 2.
device	string	Always	Device profile identifier.
name	string	Always	Configured device name.
input_count	integer	Always	Installed physical/control input count.
virtual_input_count	integer	Always	Installed virtual input count, or zero.
output_count	integer	Always	Installed controllable output count.
api_port	integer	Always	Calculated Control API TCP port.
capabilities	string[]	Always	Advertised action and feature capabilities.

Command errors: invalid_request, internal_error

1.2 get_device_info

Read stable device identity, firmware/build information and Ethernet/Wi-Fi MAC addresses.

Implemented by: All active profiles **Operation:** Read

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
device_type	string	Always	Device family/profile.
model	string	Device dependent	Hardware model or product name.

Field	Type	Presence	Description
serial_number	string	Always	Configured device serial number.
firmware_version	string	When available	Installed firmware version.
build_number	string	When available	Installed build identifier.
release_date	string	When available	Firmware release/build date.
network	object	Always	Ethernet and Wi-Fi MAC addresses.

Command errors: internal_error

1.3 get_device_state

Read the complete live snapshot used by desktop and mobile clients for synchronization.

Implemented by: All active profiles **Operation:** Read

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
revision	integer	When available	Snapshot revision/timestamp value.
captured_at	datetime	When available	Snapshot capture time.
inputs	object[]	Profiles with inputs	Current zero-based input states.
outputs	object[]	Always	Output names, states and device-specific live values.
sensors	object[]	When available	Temperature and environmental readings.
system	object	Always	Time, uptime, memory, CPU and temperature data.
network	object	Always	Current LAN/Wi-Fi addresses and SSID.
energy	object	PDU Energy Meter	Current electrical measurements.
manual_override_active	boolean	PDU profiles	Current manual override state.

Command errors: internal_error

1.4 get_relay_configuration

Read the installed input/output configuration and input-to-output mapping matrix.

Implemented by: All active profiles **Operation:** Read

Parameters

Field	Type	Required	Constraints or default	Description
-------	------	----------	------------------------	-------------

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
input_count	integer	Always	Installed input count.
output_count	integer	Always	Installed output count.
inputs	object[]	Always	Input names, enable flags and profile fields.
outputs	object[]	Always	Output names, delays, interlocks and profile fields.
mapping	object[]	When supported	Zero-based input/output mapping data.

Command errors: internal_error

2. Input output and mapping control

2.1 set_input_state

Inject a zero-based input state through the same control path used by the device input service.

Implemented by: Relay, Dimmer, PDU Energy, PDU 10 kW **Operation:** Control**Parameters**

Field	Type	Required	Constraints or default	Description
channel	integer	Yes	0 to input_count - 1	Zero-based input channel.
state	boolean	Yes	true or false	Requested input state.
source	string	No	Default Control API; max 64 chars	Audit/source description.

Expected result

Field	Type	Presence	Description
channel	integer	Always	Accepted input channel.
state	boolean	Always	Requested input state.
accepted	boolean	Always	True when queued/published.

Command errors: invalid_request, internal_error

2.2 set_virtual_input_state

Compatibility alias for input-state injection on profiles that expose virtual input control.

Implemented by: Relay, Dimmer, PDU Energy, PDU 10 kW **Operation:** Control

Parameters

Field	Type	Required	Constraints or default	Description
channel	integer	Yes	0 to input_count - 1	Zero-based input channel in the profile input space.
state	boolean	Yes	true or false	Requested virtual input state.
source	string	No	Default Control API; max 64 chars	Audit/source description.

Expected result

Field	Type	Presence	Description
channel	integer	Always	Accepted input channel.
state	boolean	Always	Requested input state.
accepted	boolean	Always	True when queued/published.

Command errors: invalid_request, internal_error

2.3 set_input_configuration

Save the editable configuration for one zero-based input channel.

Implemented by: Relay, Dimmer, PDU Energy, PDU 10 kW **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
channel	integer	Yes	0 to input_count - 1	Input channel to update.
name	string	No	Profile length rules	New display name.
enabled	boolean	No	true or false	Whether the input is enabled.

Expected result

Field	Type	Presence	Description
input	object	Always	Updated input configuration object, returned directly or as the command result.

Command errors: invalid_request, internal_error

2.4 set_output_state

Request one output to turn on or off while enforcing manual-override and configured interlocks.

Implemented by: All active profiles **Operation:** Control

Parameters

Field	Type	Required	Constraints or default	Description
channel	integer	Yes	0 to output_count - 1	Zero-based output channel.

Field	Type	Required	Constraints or default	Description
state	boolean	Yes	true or false	Requested output state.
source	string	No	Default Control API	Audit/source description when supported.

Expected result

Field	Type	Presence	Description
channel	integer	Always	Controlled output channel.
requested_state	boolean	Always	Requested logical state.
actual_state	boolean	Always	Latest known hardware state.
pending	boolean	Always	True until hardware feedback matches the request.

Command errors: invalid_request, internal_error

2.5 restart_output

Start the existing output restart operation for one zero-based output channel.

Implemented by: All active profiles **Operation:** Control

Parameters

Field	Type	Required	Constraints or default	Description
channel	integer	Yes	0 to output_count - 1	Output channel to restart.
source	string	No	Default Control API	Audit/source description.

Expected result

Field	Type	Presence	Description
channel	integer	Always	Restarted output channel.
accepted	boolean	Always	True when the restart was queued.
operation_id	string	Always	Identifier for the asynchronous restart.
off_time_ms	integer	Dimmer	Configured restart off-time used by the Dimmer path.

Command errors: invalid_request, internal_error

2.6 set_output_configuration

Save the editable configuration for one output; supported fields vary by device profile.

Implemented by: All active profiles **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
-------	------	----------	------------------------	-------------

Field	Type	Required	Constraints or default	Description
channel	integer	Yes	0 to output_count - 1	Output channel to update.
name	string	No	Profile length rules	New display name.
enabled	boolean	No	Profile dependent	Whether the output is enabled.
delay/runtime/interlock fields	mixed	No	Profile dependent	On/off delay, runtime, restart-disable, turn-off-disable and Dimmer PWM fields.

Expected result

Field	Type	Presence	Description
output	object	Always	Updated output configuration object, returned directly or as the command result.

Command errors: invalid_request, internal_error

2.7 set_mapping

Save one zero-based input-to-output mapping using the current numeric behavior code.

Implemented by: Relay, Dimmer, PDU Energy, PDU 10 kW **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
input	integer	Yes	0 to input_count - 1	Input side of the mapping.
output	integer	Yes	0 to output_count - 1	Output side of the mapping.
code	integer	Yes	Profile mapping-code range	Existing device mapping behavior code.

Expected result

Field	Type	Presence	Description
saved	boolean	Always	True after the database update was accepted.

Command errors: invalid_request, internal_error

2.8 write_hardware

Publish saved mappings and output-delay configuration to the hardware-control services.

Implemented by: Relay, Dimmer, PDU Energy, PDU 10 kW **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
hardware_write_started	boolean	Relay/PDU profiles	True when the asynchronous write begins.
hardware_write_complete	boolean	Dimmer	True when the Dimmer write request completes.

Command errors: internal_error

3. Dimmer and energy control

3.1 set_dimmer_level

Set the PWM brightness percentage for one Dimmer output channel.

Implemented by: AC/DC Dimmer **Operation:** Control

Parameters

Field	Type	Required	Constraints or default	Description
channel	integer	Yes	0 to output_count - 1	Zero-based Dimmer output channel.
value	integer	Yes	0 through 100	Requested PWM percentage.

Expected result

Field	Type	Presence	Description
channel	integer	Always	Controlled Dimmer channel.
requested_pwm	integer	Always	Requested PWM percentage.
actual_pwm	integer	Always	Latest known hardware PWM percentage.
pending	boolean	Always	True while actual PWM differs from requested PWM.

Command errors: invalid_request, internal_error

3.2 set_dimmer_frequency

Save and apply the AC/DC Dimmer frequency-mode value.

Implemented by: AC/DC Dimmer **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
value	integer	Yes	AC: 0 or 1; DC: 1, 2 or 3	Frequency mode accepted by the installed Dimmer type.

Expected result

Field	Type	Presence	Description
-------	------	----------	-------------

Field	Type	Presence	Description
pwm_frequency	integer	Always	Saved frequency-mode value.
saved	boolean	Always	True after persistence and publish.

Command errors: invalid_request, internal_error

3.3 get_output_groups

Read the AC/DC Dimmer Output Groups configuration, including the available local outputs and every saved group. Each group reports whether it is enabled and its remote retry, timeout, and failure settings. Local channels are zero-based; each remote output number is one-based.

Implemented by: AC/DC Dimmer **Operation:** Read

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
page	string	Always	output_groups.
device_family	string	Always	dimmer.
output_count	integer	Always	Number of configured local Dimmer outputs.
outputs	object[]	Always	Local channel and output_name entries.
groups	object[]	Always	Saved group objects with local and remote slaves, enabled, remote_retry_count, remote_timeout_ms and remote_failure_action.

Command errors: internal_error

3.4 save_output_group

Create or update one AC/DC Dimmer output group. The save commits the database change and publishes the complete local group-mask table to the MCU service. A remote slave stores the remote Dimmer's configured base port; PWM synchronization is sent to that port plus 1.

Implemented by: AC/DC Dimmer **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
group	object	Yes	Required	Output group to create or update.
group.id	integer or null	No	Existing positive ID; omit/null to create	Saved group identifier.

Field	Type	Required	Constraints or default	Description
group.name	string	Yes	1 to 64 characters	Unique group name, compared case-insensitively.
group.master_channel	integer	Yes	0 to output_count - 1	Zero-based local master output; unique across groups.
group.enabled	boolean	No	Default true	Enables or disables local MCU synchronization and remote Dimmer delivery for this group.
group.remote_retry_count	integer	No	0 to 10; default 2	Number of extra TCP connection/send attempts after the first attempt for each remote slave.
group.remote_timeout_ms	integer	No	100 to 10000; default 1000	TCP connect/send timeout in milliseconds for each remote attempt.
group.remote_failure_action	string	No	retry_next_feedback or hold_until_master_changes	Failure policy after all remote attempts fail. retry_next_feedback clears the cached value so the next MCU report retries it; hold_until_master_changes waits for a different master value.
group.local_slaves	integer[]	Yes	Zero-based local outputs	Local slaves; must not contain the master.
group.remote_slaves	object[]	Yes	Remote Dimmer outputs	Remote slaves. A group needs at least one local or remote slave and permits at most 32 total slaves.
remote.name	string	No	0 to 64 characters	Display name for a remote slave.
remote.ip_address	IPv4 string	Conditional	Valid IPv4 address	Remote Dimmer address; required for each remote slave.
remote.port	integer	Conditional	1 to 65534	Remote Dimmer base device port; required for each remote slave.
remote.output_number	integer	Conditional	1 to 16	One-based remote Dimmer output; required for each remote slave.

Expected result

Field	Type	Presence	Description
page	string	Always	output_groups.
device_family	string	Always	dimmer.
output_count	integer	Always	Number of configured local Dimmer outputs.
outputs	object[]	Always	Refreshed local output list.
groups	object[]	Always	Refreshed saved output groups, including enabled and remote failure settings.
saved_group_id	integer	Always	Created or updated group identifier.

Command errors: invalid_request, internal_error

3.5 delete_output_group

Delete one AC/DC Dimmer output group and all of its slave records, then publish the revised local group-mask table to the MCU service.

Implemented by: AC/DC Dimmer **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
group_id	integer	Yes	Existing positive ID	Output group to delete.

Expected result

Field	Type	Presence	Description
page	string	Always	output_groups.
device_family	string	Always	dimmer.
output_count	integer	Always	Number of configured local Dimmer outputs.
outputs	object[]	Always	Refreshed local output list.
groups	object[]	Always	Refreshed saved output groups.
deleted_group_id	integer	Always	Deleted group identifier.

Command errors: invalid_request, internal_error

3.6 rewrite_output_groups

Rewrite the complete local output-group mask table to the AC/DC Dimmer MCU. The Control API keeps the request open and returns success only after the MCU acknowledges the table.

Implemented by: AC/DC Dimmer **Operation:** Configure

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
output_group_hardware_write_complete	boolean	Always	True after the MCU acknowledges the complete table.

Command errors: internal_error

3.7 get_energy_state

Read the current PDU Energy Meter electrical measurements without converting missing readings to zero.

Implemented by: PDU Energy Meter **Operation:** Read

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
energy	object	Always	Energy measurement object.
energy.voltage_v	number or null	Always	Voltage reading.
energy.current_a	number or null	Always	Current reading.
energy.active_power_w	number or null	Always	Active power.
energy.apparent_power_va	number or null	Always	Apparent power.
energy.power_factor	number or null	Always	Power factor.
energy.energy_kwh	number or null	Always	Accumulated energy.
energy.available	boolean	Always	True when every required reading is available.

Command errors: internal_error

3.8 get_energy_history

Read down-sampled historical Energy Meter points for one measurement and date range.

Implemented by: PDU Energy Meter **Operation:** Read

Parameters

Field	Type	Required	Constraints or default	Description
parameter	integer	Yes	0 voltage, 1 current, 2 power, 3 PF, 4 VA, 5 energy	Measurement selector.
start_date	date	Yes	YYYY-MM-DD	Inclusive start date.
end_date	date	Yes	YYYY-MM-DD; maximum 365 days	Inclusive end date.

Expected result

Field	Type	Presence	Description
energy_history	boolean	Always	True for an energy-history result.
parameter	integer	Always	Echoed measurement selector.
unit	string	Always	Measurement unit.
start_date	date	Always	Echoed start date.
end_date	date	Always	Echoed end date.

Field	Type	Presence	Description
points	object[]	Always	Timestamp/value samples, limited by down-sampling.

Command errors: invalid_request, internal_error

4. Configuration pages

4.1 get_page_configuration

Read a supported UI/configuration page using the device's existing page schema.

Implemented by: All active profiles **Operation:** Read

Parameters

Field	Type	Required	Constraints or default	Description
page	string	Yes	Profile-specific page name	Page to read.
log_page	integer	No	System page; default 1	One-based system-log page.
log_page_size	integer	No	System page; profile limits	Number of log rows per page.

Expected result

Field	Type	Presence	Description
page	object	Always	Page-specific sections, fields, columns, rows and optional pagination.

Command errors: invalid_request, internal_error

4.2 set_page_configuration

Save one supported section of a device configuration page.

Implemented by: All active profiles **Operation:** Configure or admin

Parameters

Field	Type	Required	Constraints or default	Description
page	string	Yes	Profile-specific page name	Page containing the section.
section	string	Yes	Supported section name	Section to update.
values	object	Yes	Section-specific fields	Values to validate and persist.

Expected result

Field	Type	Presence	Description
page	object	Always	Updated page or section result.

Command errors: invalid_request, internal_error

4.3 mutate_page_row

Add, update or delete one row in a supported schedule, automation or security table.

Implemented by: All active profiles **Operation:** Configure or admin

Parameters

Field	Type	Required	Constraints or default	Description
page	string	Yes	Profile-specific page name	Page containing the row collection.
section	string	Yes	Supported row section	Row collection to mutate.
operation	string	Yes	add, update or delete	Requested row operation.
id	integer	Update/delete	Database row ID	Existing row identifier.
values	object	Add/update	Section-specific fields	Row values to validate and persist.

Expected result

Field	Type	Presence	Description
page	object	Always	Updated page/row collection after the mutation.

Command errors: invalid_request, internal_error

4.4 execute_page_action

Execute an implemented page operation such as time synchronization, NTP test, log deletion or reboot.

Implemented by: All active profiles **Operation:** Configure or admin

Parameters

Field	Type	Required	Constraints or default	Description
page_action	string	Yes	Profile-specific implemented action	Operation to execute.
datetime	string	Conditional	YYYY-MM-DD HH:MM:SS	Client/manual time for supported time actions.
server	string	Conditional	Host name or IP	NTP server to test.
log_page_size	integer	No	delete_system_logs	Requested size of the refreshed system-log page.
action-specific fields	mixed	Conditional	Dimmer sequence/scenario rewrites	Additional values required by the selected page action.

Expected result

Field	Type	Presence	Description
-------	------	----------	-------------

Field	Type	Presence	Description
result	object	Always	Action acknowledgment, test result or refreshed page data.

Command errors: invalid_request, internal_error

5. Backup restore and transfers

5.1 transfer_upload_begin

Create or extend a chunked upload session for settings, credentials or firmware files supported by the profile.

Implemented by: All active profiles **Operation:** Admin

Parameters

Field	Type	Required	Constraints or default	Description
kind	string	Yes	Profile-supported transfer kind	Upload/commit workflow.
role	string	Yes	Kind-specific file role	Role of this file in the transfer.
filename	string	Yes	Safe base filename	Original file name.
size	integer	Yes	Profile size limit	Expected file size in bytes.
transfer_id	string	No	Existing multi-file transfer	Attach another role to an existing transfer.

Expected result

Field	Type	Presence	Description
transfer_id	string	Always	Upload-session identifier.
role	string	Always	Accepted file role.
size	integer	Always	Expected total file size.
received	integer	Always	Bytes currently received.
chunk_size	integer	Always	Maximum/recommended chunk size.

Command errors: invalid_request, internal_error

5.2 transfer_upload_chunk

Append one base64-encoded byte chunk at the required upload offset.

Implemented by: All active profiles **Operation:** Admin

Parameters

Field	Type	Required	Constraints or default	Description
-------	------	----------	------------------------	-------------

Field	Type	Required	Constraints or default	Description
transfer_id	string	Yes	Open upload session	Transfer identifier.
role	string	Yes	Role opened by begin	File role.
offset	integer	Yes	Exact next byte offset	Chunk byte offset.
data	base64 string	Yes	Within negotiated chunk size	Chunk bytes.

Expected result

Field	Type	Presence	Description
received	integer	Always	Total bytes received for this role.
size	integer	Always	Expected total file size.

Command errors: invalid_request, internal_error

5.3 transfer_upload_finish

Finish and validate one uploaded file role before commit.

Implemented by: All active profiles **Operation:** Admin

Parameters

Field	Type	Required	Constraints or default	Description
transfer_id	string	Yes	Open upload session	Transfer identifier.
role	string	Yes	Completed file role	Role to finish.

Expected result

Field	Type	Presence	Description
complete	boolean	Always	True after size and file validation succeeds.
role	string	Always	Completed role.
digest	string	When provided	Calculated file digest.

Command errors: invalid_request, internal_error

5.4 transfer_commit

Install or restore a validated upload and report whether the device should reboot.

Implemented by: All active profiles **Operation:** Admin

Parameters

Field	Type	Required	Constraints or default	Description
transfer_id	string	Yes	Completed upload session	Transfer to commit.

Expected result

Field	Type	Presence	Description
message	string	Always	Commit result description.
reboot_now	boolean	Always	True when firmware schedules an immediate reboot.

Command errors: invalid_request, internal_error

5.5 settings_backup_download_begin

Create a temporary settings-database snapshot and start a chunked download session.

Implemented by: All active profiles **Operation:** Admin

Parameters

Field	Type	Required	Constraints or default	Description
(none)	-	-	-	This action has no action-specific parameters.

Expected result

Field	Type	Presence	Description
transfer_id	string	Always	Download-session identifier.
size	integer	Always	Snapshot size in bytes.
chunk_size	integer	Always	Maximum/recommended read size.
digest	string	When provided	Snapshot digest.

Command errors: internal_error

5.6 transfer_download_chunk

Read one base64-encoded data chunk from an open download session.

Implemented by: All active profiles **Operation:** Admin

Parameters

Field	Type	Required	Constraints or default	Description
transfer_id	string	Yes	Open download session	Transfer identifier.
offset	integer	Yes	Valid byte offset	First byte to read.

Expected result

Field	Type	Presence	Description
data	base64 string	Always	Returned bytes.
offset	integer	Always	Echoed byte offset.
size	integer	Always	Total download size.

Field	Type	Presence	Description
complete	boolean	Always	True when this chunk reaches the end.

Command errors: invalid_request, internal_error

5.7 transfer_download_finish

Close a download session and remove its temporary snapshot.

Implemented by: All active profiles **Operation:** Admin

Parameters

Field	Type	Required	Constraints or default	Description
transfer_id	string	Yes	Open download session	Transfer identifier.

Expected result

Field	Type	Presence	Description
complete	boolean	Always	True after cleanup.

Command errors: invalid_request, internal_error

Shared data schemas

These compact definitions identify the fields expected in repeated result objects. A later JSON Schema/OpenAPI file can make them machine-validatable without changing the command contract.

Type	Required/core fields
DeviceInfo	device_id, name, device_type, model, serial_number, firmware_version, hardware_version
Topology	input_count, virtual_input_count, output_count, sensor_count
InputState	kind, channel, name, enabled, state, mode, active_level, debounce_ms, changed_at
OutputState	channel, name, enabled, state, pending, requested_level?, actual_level?, configuration?, changed_at
OutputConfiguration	channel, name, enabled, delays/runtimes in ms, initial_state, restart_off_time_ms
Mapping	input, output, behavior, legacy_code; codes: 0 none, 1 on, 2 off, 3 toggle, 4 continuous_on, 5 continuous_off
Schedule	id, revision, name, enabled, type, timing, actions, last_run, next_run
Schedule timing	one_time: datetime; recurring: time, weekdays[0=Monday..6=Sunday], optional start/end dates
Action	action plus params; only capability-advertised control actions are permitted
Automation	id, revision, name, enabled, type, trigger configuration and actions
Scenario	id, revision, name, default_transition_ms and targets[channel,state,level]

Type	Required/core fields
Sequence	id, revision, name, steps[scenario_id or actions, hold_ms], loop_count
TemperatureReading	sensor_id, name, value_c, status, timestamp
ChannelEnergy	channel, voltage_v?, current_a?, active_power_w?, apparent_power_va?, power_factor?, energy_wh?, timestamp
AccessRule	id, service, kind, value, name, enabled, revision
Fault	fault_id, severity, code, message, active, first_seen, last_seen

Configuration section schemas

Section	Fields
general	name, location?, description?, language?, physical_input_count (read-only), output_count (read-only)
account	username, password (write-only), session_timeout_s, password_change_required
lan	mode[dhcp static], address, subnet_mask, gateway, dns_servers[], control_port = legacy_port + 3 (default 5008), legacy_port default 5005
wifi	enabled, mode[dhcp static], ssid, password (write-only), address, subnet_mask, gateway, dns_servers[]
time	timezone, sync_mode[manual ntp], ntp_server, sync_interval_s
temperature	enabled, low_warning_c, high_warning_c, control/hysteresis fields where supported
tcp_access	tcp_enabled, legacy_port, allow_filter_enabled; tcp_enabled controls legacy_port, legacy_port + 1 and Control API TCP legacy_port + 3; rules are managed separately
api_access	api_enabled, IP/MAC allow rules, HTTP enabled, HTTPS enabled, authentication target, rate-limit target; api_enabled controls HTTP/HTTPS command endpoints only
mqtt	enabled, host, port, tls, username, password(write-only), client_id, base_topic, qos
tls	enabled, certificate metadata, key_present, minimum_tls_version; key material is write-only via upload
maintenance	log_level, automatic_backup settings, retention, update channel where supported

Device events

Events are unsolicited messages on subscribed TCP/WebSocket sessions or SSE streams. They use protocol, event, subscription_id and data fields; they do not contain ok or result.

```
{"protocol":3,"event":"output_state_changed","subscription_id":"sub-7","data":{"channel":0,"previous_state":false,"state":true,"pending":false,"source":"windows-app","revision":312,"timestamp":"2026-08-31T10:20:30+00:00"}}
```

Event	Expected data fields
output_state_changed	channel, previous_state, state, pending, source, revision, timestamp
output_level_changed	channel, requested_level, actual_level, transitioning, source, revision, timestamp
input_state_changed	kind, channel, previous_state, state, source, revision, timestamp
mapping_changed	input, output, behavior, legacy_code, revision, timestamp
temperature_changed	sensor_id, value_c, status, timestamp
energy_changed	summary and/or channel readings, timestamp
schedule_executed	schedule_id, results, success, timestamp
automation_executed	automation_id, trigger, results, success, timestamp
sequence_state_changed	sequence_id, operation_id, state, step, loop, timestamp
operation_progress	operation_id, kind, stage, progress_percent, message, timestamp
device_fault	fault_id, severity, code, message, active, timestamp
configuration_changed	section, revision, source, restart_required, timestamp
device_rebooting	reason, reboot_in_ms, timestamp

Synchronization: Clients must treat events as incremental updates. If revisions are skipped, reconnecting clients should call `get_device_state` to rebuild a complete state snapshot.

Migration and acceptance criteria

- Migrated Relay Module, AC/DC Dimmer and PDU firmware exposes its implemented protocol 2 command subset on configured base port + 3 (5008 by default).
- Each migrated firmware family exposes the same implemented commands through POST /api/v1/command on HTTP or HTTPS.
- Discovery and heartbeat identity report the calculated Control API port so clients do not hardcode 5008.
- The standard Windows paths for all five migrated families use the JSON Control API and do not send AT commands or use the base port.
- Each configured base port remains AT-only for existing third-party or service integrations.
- Each migrated screen is tested against Relay Module, AC/DC Dimmer, PDU Energy Meter, PDU 10 kW and PDU V1 capabilities as applicable.
- Every active firmware family creates tcp_enabled and api_enabled automatically during database migration, defaults both to enabled, and persists later changes.
- With tcp_enabled false, base TCP, base + 1 and base + 3 are closed after restart while discovery, heartbeat and the web UI remain active.
- With api_enabled false, HTTP/HTTPS command routes return 503 api_disabled while raw Control API TCP and the web UI remain available.
- The mobile application uses the Control API envelope and relies on hello/capability data instead of a fixed port or device-model assumptions.
- Automated tests verify TCP framing, invalid JSON, HTTP status mapping, 1 MiB size limits, allow rules, index bounds, concurrency and reconnection.

The implemented-only catalogue must be updated whenever a new action is added to firmware dispatch, hello capabilities, the Windows/mobile client path and its automated contract tests.